

Debugger une application en cours d'exécution

Debugger une application en cours d'exécution

Objectifs du module

À la fin de ce module, vous serez capable de :

- Attacher strace à un processus live et filtrer la trace utile
- Distinguer un appel système d'un appel de bibliothèque
- Reconnaître un processus bloqué et trouver le syscall responsable
- Choisir le bon signal pour intervenir sans casser l'état

Plan

1. strace en profondeur
2. ltrace et inspection des libs
3. Signaux et processus hung
4. Démo et TP
5. Récap et questions

1. strace en profondeur

Ce que montre strace

“ strace trace les appels système faits par un processus et leurs retours. ”

Points essentiels :

- Voit la frontière user / kernel
- N'a pas besoin de recompiler l'application
- Ralentit le processus tracé, parfois beaucoup
- Ne voit pas les appels internes à l'application

Invocations utiles

```
# Lancer un binaire sous trace
strace -f -o trace.log ./monbin

# Attacher à un processus existant
strace -f -p <pid>

# Filtrer par famille de syscalls
strace -e trace=network -p <pid>
strace -e trace=openat,read,write -p <pid>

# Statistiques par syscall
strace -c -p <pid>
```

`-f` suit les fork ; sans `-f` vous ratez les threads et enfants.

Lire une trace

```
openat(AT_FDCWD, "/etc/hosts", O_RDONLY) = 5
read(5, "127.0.0.1 localhost\n...", 4096) = 234
close(5) = 0
connect(7, {sa_family=AF_INET, ...}, 16) = -1 EINPROGRESS
```

- Valeur de retour négative = erreur ; EINPROGRESS, EAGAIN, ENOENT sont fréquents
- Un syscall sans retour visible = bloqué, c'est souvent votre coupable

2. Itrace et inspection des libs

strace vs ltrace

strace

- Appels système uniquement
- Frontière user/kernel
- Toujours dispo
- Idéal : freeze I/O, permissions

ltrace

- Appels de bibliothèques dynamiques
- Niveau au-dessus
- Plus coûteux, parfois instable
- Idéal : comprendre ce qu'utilise un binaire

Code et commandes

```
# Voir les appels libc d'un process
ltrace -p <pid>

# Filtrer sur malloc/free
ltrace -e 'malloc+free' -p <pid>

# Tracer l'enchaînement lib + syscalls
ltrace -S ./monbin
```

Attention : ltrace ne marche pas sur les binaires statiquement liés.

3. Signaux et processus hung

Diagnostiquer un freeze

Démarche :

1. Lire `/proc/<pid>/status` et noter l'état (R, S, D, Z)
2. Si D persistant : regarder `/proc/<pid>/stack` (nécessite root) et `/proc/<pid>/wchan`
3. Si S : attacher strace pour voir le syscall en attente
4. Si boucle R : passer au profiling (module 5)

Bon usage des signaux

- Préférer `SIGTERM` puis `SIGKILL` en dernier recours
- `SIGQUIT` sur une JVM déclenche un thread dump
- `SIGHUP` recharge la config de beaucoup de démons
- `SIGSTOP` suspend sans tuer, utile pour capturer un état

```
kill -SIGTERM <pid>
kill -SIGQUIT <pid>    # JVM thread dump
kill -SIGSTOP <pid>    # geler pour inspection
kill -SIGCONT <pid>    # reprendre
```

TP à suivre

TP 1 : investiguer une application qui freeze et un scénario de crash

- Durée : 1h
- Fichier : `tp/tp-01.md`

